

SAKARYA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

PicoRV İşlemci Alt Kümesi (RV32I) için FPGA tabanlı Loader Tasarımı

Proje Raporu

Öğrenci Adı Soyadı:	GEYDANUR TUBA YILMAZ - OĞUZHAN ÖZEN
Öğrenci Numarası:	23010903086 - 22010903114
Ders:	SİSTEM PROGRAMLAMA
Öğretim Görevlisi:	PROF. DR. HALİT ÖZTEKİN
Tarih:	HAZİRAN 2026

2025/2026 BAHAR

Özet

Bu çalışmada, Tang Nano 9K FPGA kartı üzerinde çalışan PicoRV32 tabanlı bir sistem için adres-bağımsız assembler, GNU-benzeri linker-script destekli linker ve güvenilir UART program yükleyici geliştirilmiştir. İki geçişli assembler, RV32I komutlarını ve section tabanlı Assembly kaynaklarını nihai adres içermeyen object dosyalarına dönüştürmektedir. Linker; sembolleri çözmekte, relocation kayıtlarını uygulamakta, section'ları 16 KiB BRAM sınırları içine yerleştirmekte ve hedef adresleri taşıyan tek bir .picoimg yükleyici görüntüsü üretmektedir. Host yazılımı bu görüntüyü CRC32 korumalı, sıra numaralı ve ACK/NACK geri bildirimli UART paketleriyle FPGA üzerindeki yükleyici durum makinesine aktarmaktadır.

Sistem dört farklı Assembly senaryosuyla doğrulanmıştır: Walking-One Pattern Test, Up/Down Counter Test, Multi-Object Call/Relocation Test ve Memory Boundary/Out-of-Range Test. COM7 üzerinden 115.200baud hızında yapılan on tekrarlı ölçümlerde ortalama yükleme süreleri 22,364ms ile 35,805ms arasında ölçülmüştür. Gowin yerleştirme ve yönlendirme sonuçlarında 2095 LUT (%24,25), 965 register (%14,42), 15 BSRAM (%57,69) ve toplam 2695 logic birimi (%31,19) kullanılmış; azami çalışma frekansı 42,442MHz olarak elde edilmiştir. Sonuçlar, eğitim amaçlı açık bir araç zincirinde adres yerleştirme, güvenilir veri aktarımı ve donanım-yazılım ortak tasarımının uçtan uca gerçekleştirilebildiğini göstermektedir.

Anahtar Kelimeler: RISC-V, PicoRV32, FPGA, assembler, linker, relocation, UART, CRC32, program yükleyici.

İçindekiler

Özet	i
Kısaltmalar	v
1 Giriş ve Literatür Araştırması	1
1.1 Problem Tanımı ve Amaç	1
1.2 Program Yükleme Mimarileri	1
1.3 Veri Bütünlüğü ve Hata Doğrulama	2
1.4 RISC-V, PicoRV32 ve Açık Araç Zinciri	3
1.5 Seçilen Mimarinin Gerekçesi	3
2 Sistem Tasarımı ve Gerçekleştirme	4
2.1 Genel Sistem Mimarisi	4
2.2 Assembler Tasarımı	4
2.3 Linker ve Bellek Yerleştirme	5
2.4 .picoimg Yükleyci Görüntüsü	6
2.5 UART Host Loader ve Protokol	6
2.6 FPGA Loader FSM, İşlemci ve MMIO	6
2.7 Tkinter IDE İş Akışı	7
3 Deneysel Çalışmalar, Test ve Analiz	8
3.1 Deney Tasarımı ve Test Senaryoları	8
3.1.1 Walking-One Pattern Test	9
3.1.2 Up/Down Counter Test	9
3.1.3 Multi-Object Call/Relocation Test	9
3.1.4 Memory Boundary/Out-of-Range Test	9
3.2 Veri Toplama ve Donanım Metrikleri	10
3.2.1 Yükleme Süresi Sonuçlarının Analizi	11
3.2.2 FPGA Kaynak Tüketimi ve Zamanlama	11
4 Projenin Küresel, Toplumsal ve Ekonomik Etkileri	13
4.1 Sürdürülebilirlik ve Yeşil Bilişim	13
4.2 Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık	13

4.3 Fonksiyonel Güvenlik ve Sağlık	14
4.4 E-Atık Yönetimi ve Döngüsel Ekonomi	14
5 Proje Yönetimi ve Takım Çalışması	15
5.1 Görev Dağılımı ve Sorumluluk Matrisi	15
5.2 Koordinasyon ve Sürüm Kontrol Yönetimi	15
6 Bireysel Katkı Beyanı	17
7 Sonuç ve Gelecek Çalışmalar	19
7.1 Sınırlamalar	19
8 Kaynakça	20
A Program Öğrenme Çıktıları	22
B UART Komutları ve Hata Kodları	24
C Tekrar Üretim ve Test Kodları	25
C.1 Walking-One Pattern Test Kodları	25
C.2 Up/Down Counter Test Kodları	26
C.3 Multi-Object Call/Relocation Test Kodları	28
C.4 Memory Boundary/Out-of-Range Test Kodları	29

Şekil Listesi

2.1 Genel sistem mimarisi	4
2.2 Assembler ve linker veri akışı	5
2.3 .picoimg dosya düzeni	6
2.4 UART paket yapısı	6
2.5 FPGA loader durum makinesinin sadeleştirilmiş görünümü	7
2.6 PicoRV32 bellek ve MMIO haritası	7
3.1 CSV verisinden üretilen ortalama UART yükleme süreleri	10
3.2 .picoimg boyutu ile ortalama yükleme süresi ilişkisi	11
3.3 FPGA kaynak kullanım yüzdeleri	12

Tablo Listesi

1.1 Program yükleme mimarilerinin karşılaştırılması	2
1.2 Bütünlük ve geri bildirim yöntemlerinin karşılaştırılması . . .	3
3.1 Test senaryolarının kapsamı	8
3.2 Senaryo bazlı program ve UART yükleme metrikleri	10
3.3 Gowin PNR donanım kaynakları	11
5.1 Proje RACI matrisi	15
A.1 Program Öğrenme Çıktılarının tam listesi	22
B.1 UART protokol komutları	24
B.2 UART protokol hata durumları	24

Kısaltmalar

ACK	Acknowledgement, olumlu alındı bildirimi
ARQ	Automatic Repeat reQuest, otomatik tekrar isteği
BRAM/BSRAM	FPGA içi blok bellek
CRC	Cyclic Redundancy Check
FSM	Finite State Machine, sonlu durum makinesi
FPGA	Field-Programmable Gate Array
ISA	Instruction Set Architecture
JTAG	Joint Test Action Group hata ayıklama/programlama arayüzü
LUT	Look-Up Table
MMIO	Memory-Mapped Input/Output
NACK	Negative Acknowledgement, olumsuz alındı bildirimi
PÇ	Program Öğrenme Çıktısı
PNR	Place and Route, yerleştirme ve yönlendirme
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver/Transmitter

Bölüm 1

Giriş ve Literatür Araştırması

1.1 Problem Tanımı ve Amaç

Soft-core işlemci içeren bir FPGA sisteminde Assembly kaynak kodunun fiziksel olarak çalıştırılabilmesi için yalnızca makine koduna çevrilmesi yeterli değildir. Komut ve verilerin bellekteki nihai adresleri belirlenmeli, sembolik başvurular çözümlenmeli, çıktı hedef belleğe güvenilir biçimde aktarılmalı ve aktarım tamamlanmadan işlemci çalıştırılmamalıdır. Bu çalışmada, bu adımların tamamını görünür ve incelenebilir hâle getiren eğitim amaçlı bir araç zinciri tasarlamayı amaçlamaktadır. Bu süreç, bilgisayar mimarisi ve sistem programlama bilgisinin uygulanmasını desteklemektedir (PÇ1).

Çalışmanın temel katkıları aşağıdaki gibidir:

- Section, sembol ve relocation bilgisi taşıyan adres-bağımsız object biçimi,
- GNU-benzeri linker script ile denetlenebilir bellek yerleşimi,
- Hedef adresler ve bütünlük bilgisi taşıyan tek dosyalı .picoimg biçimi,
- CRC32, sequence, ACK/NACK, timeout ve retry mekanizmaları içeren UART protokolu,
- PicoRV32, 16 KiB BRAM, loader FSM ve MMIO çevre birimlerini birleştiren FPGA sistemi,
- Gerçek kart üzerinde tekrarlı deneyler ve kaynak tüketimi ölçümleri.

1.2 Program Yükleme Mimarileri

Gömülü sistemlerde program yükleme için UART, SPI, JTAG ve Boot ROM tabanlı yaklaşımlar yaygın olarak kullanılmaktadır [4]. Gömülü yazılım geliştirme açısından bu yaklaşımların temel özellikleri Valvano tarafından da ele alınmıştır [5]. Sistem düzeyindeki donanım-yazılım ortak tasarımı Black ve diğerleri tarafından incelenmiştir [6]. Veri iletişimi açısından seri aktarım ilkeleri Stallings'in çalışmasıyla ilişkilidir [9]. JTAG sınır tarama mimarisi IEEE 1149.1 standardında tanımlanmıştır [7]. Üretici uygulama ayrıntıları ise teknik başvuru dokümanlarında açıklanmaktadır [8]. UART, düşük pin sayısı ve USB-UART dönüştürücülerle kolay erişim sağlaması nedeniyle

prototiplemede elverişlidir. SPI daha yüksek veri hızına ulaşabilse de saat hattı, chip-select yönetimi ve çoğunlukla harici flash gerektirir. JTAG, sınır tarama ve hata ayıklama için güçlüdür; ancak kullanıcı uygulamasını çalışma zamanında yükleyen sade bir eğitim protokolü için gereğinden karmaşık olabilir. Boot ROM/FSBL yaklaşımı kalıcı ve bağımsız açılış sağlarken değişmez bir başlangıç yazılımı ve flash yönetimi gerektirir.

Tablo 1.1: Program yükleme mimarilerinin karşılaştırılması

Yaklaşım	Avantaj	Sınırlama	Ek donanım	Bu proje açısından
UART	Basit, düşük pin sayısı, kolay gözlem	Görece düşük hız	USB-UART	Seçilmiştir
SPI/Flash	Yüksek hız, kalıcı açılış	Flash ve denetleyici karmaşıklığı	Harici/yerleşik flash	Gelecek çalışma
JTAG	Programlama ve hata ayıklama	Üretici aracı ve özel akış	JTAG kablo-su/çekirdeği	Bitstream yükleme için
Boot ROM	Bağımsız ve otomatik açılış	Sabit ROM ve boot zinciri	ROM/flash	Proje kapsamı dışında

Bu projede FPGA bitstream'i Gowin Programmer aracılığıyla SRAM'e bir kez yüklenmekte; ardından farklı kullanıcı programları, güç kesilmediği sürece UART üzerinden art arda BRAM'e aktarılabilir. Böylece üreticiye özgü FPGA programlama işlemi ile kullanıcı programının çalışma zamanı yüklenmesi birbirinden ayrılmıştır.

1.3 Veri Bütünlüğü ve Hata Doğrulama

Seri iletişimde bit hataları, eksik paketler, tekrarlanan paketler ve gecikmeler dikkate alınmalıdır. Parite tek bit hatalarının bir bölümünü yakalarken checksum yöntemleri belirli hata örüntülerinde zayıf kalabilir. CRC, polinom bölme temelli yapısı sayesinde patlama hatalarına karşı güçlü bir tespit mekanizmasıdır. CRC'nin matematiksel hata tespit özellikleri Peterson ve Brown tarafından gösterilmiştir [11]. Ağ protokollerindeki bütünlük yaklaşımı RFC 791'de örneklenmektedir [10]. Güvenilir veri aktarımının geri bildirim ve tekrar mekanizmaları Tanenbaum ve Wetherall tarafından açıklanmaktadır [12]. Ancak CRC yalnızca hatayı tespit eder; güvenilir aktarım için ACK/NACK, sequence numarası, timeout ve tekrar gönderim mekanizmalarıyla birlikte kullanılmalıdır.

Tablo 1.2: Bütünlük ve geri bildirim yöntemlerinin karşılaştırılması

Yöntem	Tespit yeteneği	Maliyet	Projedeki kullanım
Parite	Sınırlı tek-bit hata tespiti	Çok düşük	Kullanılmıyor
Checksum	Basit toplamsal hata tespiti	Düşük	Kullanılmıyor
CRC32	Güçlü rastlantısal/patlama hata tespiti	Orta	Paket, segment ve image düzeyinde
ACK/NACK + ARQ	Teslimat ve tekrar kontrolü	Ek yanıt trafiği	Her komut paketinde

1.4 RISC-V, PicoRV32 ve Açık Araç Zinciri

RISC-V, açık ve genişletilebilir bir komut kümesi mimarisidir [1]. Sembol bağlama, çağrı kuralları ve relocation işlemleri RISC-V psABI belgesinde tanımlanmıştır [2]. Yapılan literatür incelemesi, mimari kararların resmi dokümanlar ve akademik çalışmalarla gerekçelendirilmesini sağlamıştır (PÇ6). RV32I tabanı, öğretim amaçlı bir assembler ve linker için yeterli aritmetik, mantıksal, bellek, dallanma ve çağrı komutlarını içerir. PicoRV32 ise Clifford Wolf tarafından geliştirilen, alan verimliliği odaklı açık kaynak bir RISC-V çekirdeğidir [3]. Küçük FPGA’larda uygulanabilmesi ve basit bellek arayüzü, bu çalışma için uygun bir işlemci çekirdeği olmasını sağlamıştır.

1.5 Seçilen Mimarinin Gerekçesi

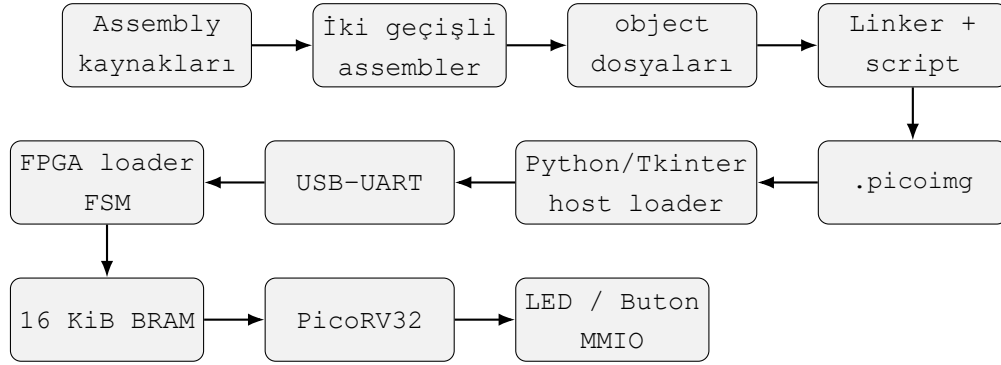
UART-loader mimarisi; gözlenebilir paket akışı, düşük bağlantı maliyeti, Python ile kolay host uygulaması geliştirme ve farklı programların aynı bitstream üzerinde hızla denenebilmesi nedeniyle seçilmiştir. CRC32 ve ARQ birleşimi iletişim güvenilirliğini; linker script ve relocation kayıtları ise adres yerleştirmenin doğruluğunu sağlar. Bu seçim, endüstriyel bir boot zincirinin tüm özelliklerini hedeflememekte; bunun yerine sistem programlama kavramlarını uçtan uca görünür kılmaktadır.(PÇ6)

Bölüm 2

Sistem Tasarımı ve Gerçekleştirme

2.1 Genel Sistem Mimarisi

Şekil 2.1, kaynak koddan fiziksel LED ve buton davranışına kadar sistemin genel mimarisini göstermektedir.



Şekil 2.1: Genel sistem mimarisi

2.2 Assembler Tasarımı

Assembler iki geçişli olarak çalışmaktadır. İlk geçişte section konum sayacıları, etiketler ve direktiflerin boyut etkileri belirlenir. İkinci geçişte komutlar kodlanır; henüz nihai adresi bilinmeyen sembolik başvurular için relocation kayıtları oluşturulur. Bu ayırım, assembler'ın bellek haritasından bağımsız kalmasını ve farklı linker script'leriyle aynı object dosyalarının kullanılmasını sağlar.

JSON object biçimi; sections, symbols ve relocations alanlarından oluşur. PROGBITS section'ları gerçek byte içeriği taşırken NOBITS section'ları yalnızca çalışma zamanı boyutunu taşır. .text, .rodata, .data, .bss ve özel .section tanımları desteklenmektedir. HI20, LO12-I, LO12-S, BRANCH ve JAL relocation türleri, RISC-V psABI yaklaşımıyla uyumlu anlamlara sahiptir [2].

Kod Bloğu 2.1: Sembol ve relocation gerektiren kısa Assembly örneği

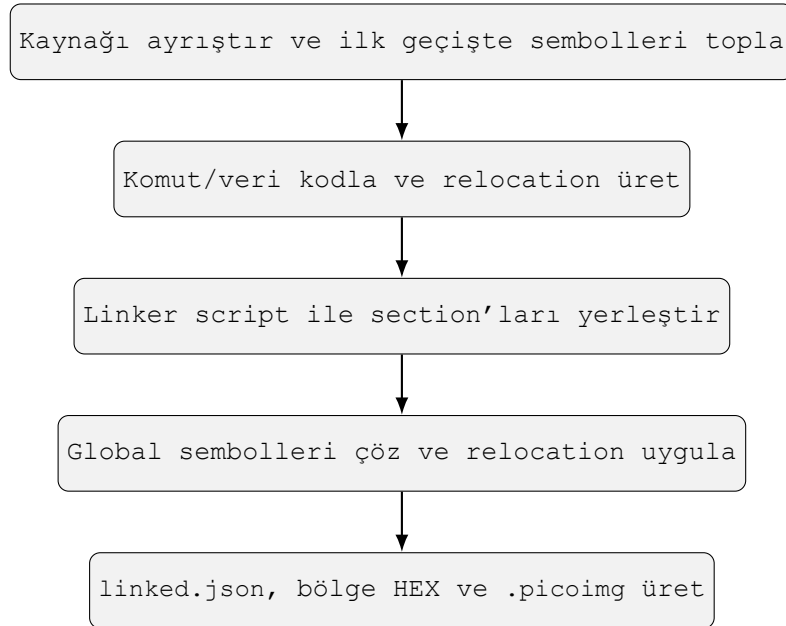
```

1 .text
2 .global _start
3 .extern delay_start
4 _start:
5     lui    x5, led_addr
6     addi   x6, x0, 1
7     sw     x6, led_addr(x5)
8     jal    x1, delay_start
9     jal    x0, _start

```

2.3 Linker ve Bellek Yerleştirme

Linker, bir veya daha fazla object dosyasını zorunlu bir linker script ile birleştirir. Script; ENTRY, MEMORY ve SECTIONS tanımlarını kullanarak giriş sembolünü, fiziksel bellek bölgelerini ve input section'ların output section'lara yerleşimini belirler. Global sembol çözümleme sonrasında relocation kayıtları gerçek adreslerle yamalanır.

**Şekil 2.2:** Assembler ve linker veri akışı

Linker; orphan section, bellek bölgesi taşması, izin uyumsuzluğu, çakışma, tanımsız sembol, global olmayan entry ve dallanma/çağrı erişim aralığı hatalarını raporlar. Bu kontroller, hatalı bir programın karta gönderilmeden önce durdurulmasını sağlar.

2.4 .picoimg Yükleyici Görüntüsü

Ham bir .bin yalnızca ardışık byte dizisi taşır ve hedef adresi hakkında bilgi vermez. .picoimg, adres bilgisi ile bütünlük denetimini aynı dosyada birleştirir. Tüm çok baytlı alanlar little-endian kodlanır. Image CRC32, descriptor tablosu ile payload'ın tamamını; segment CRC32 ise her segmentin kendi verisini korur.

.picoimg biçimi, ileride farklı başlangıç adreslerini destekleyebilmek için entry alanı taşımaktadır. Ancak mevcut FPGA entegrasyonunda PicoRV32, reset sonrasında 0x00000000 adresinden başlatılmaktadır ve test programları bu adrese yerleştirilmiştir. Dolayısıyla format düzeyinde entry bilgisi bulunsa da mevcut donanım hedefindeki fiili başlangıç adresi 0x00000000'dır.

Header 32 byte	Segment descriptor'ları 16 byte / segment	Payload segment verileri
-------------------	--	-----------------------------

magic, version, entry, address, length, flags, BRAM'e yazılacak
segment count, image CRC32 data CRC32 byte dizileri

Şekil 2.3: .picoimg dosya düzeni

2.5 UART Host Loader ve Protokol

Host loader, .picoimg dosyasını önce yerel olarak doğrular ve yüklenebilir segmentleri en fazla 128 byte payload taşıyan DATA paketlerine böler. Bir yükleme oturumu PING, BEGIN, bir veya daha fazla DATA, END ve RUN komutlarından oluşur. Varsayılan hat ayarı 115.200baud, 8N1, 1s timeout ve paket başına üç denemedir.

Magic 2 B	Ver. 1 B	Type 1 B	Seq. 1 B	Flags 1 B	Length 2 B	Address 4 B	Payload 0-128 B	CRC32 4 B
--------------	-------------	-------------	-------------	--------------	---------------	----------------	--------------------	--------------

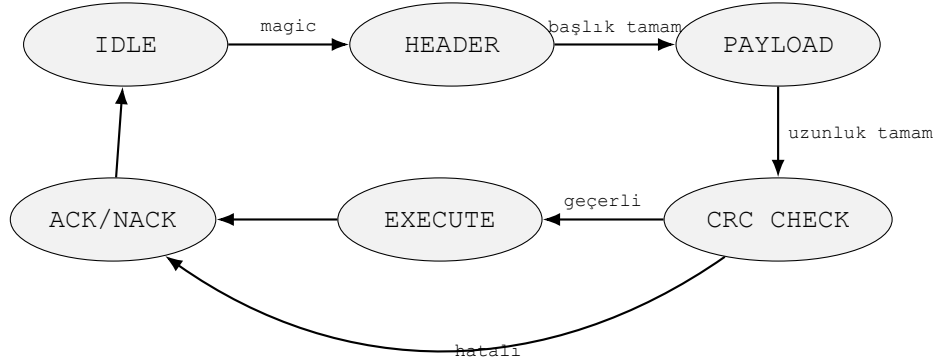
Şekil 2.4: UART paket yapısı

Her komut yalnızca aynı sequence numarasını taşıyan doğru ACK alındığında tamamlanmış sayılır. Timeout, bozuk yanıt veya NACK durumunda aynı paket aynı sequence değeriyle tekrar gönderilir. FPGA, bir önceki kabul edilmiş sequence tekrarlandığında işlemi ikinci kez uygulamak yerine önceki ACK'i yeniden üretir. Bu idempotent davranış, ACK kaybının çift BRAM yazmasına yol açmasını önler.

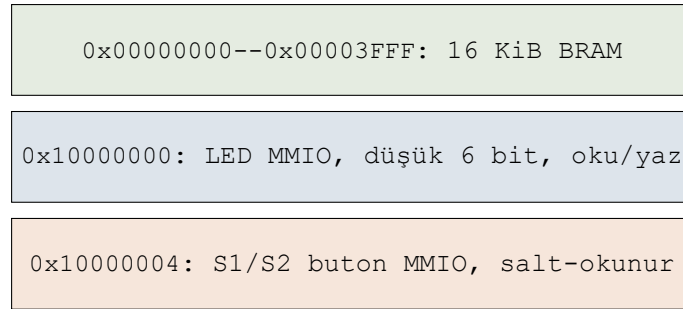
2.6 FPGA Loader FSM, İşlemci ve MMIO

FPGA loader, UART byte akışını durum makinesiyle ayrıştırır. Paket başlığı ve payload alındıktan sonra CRC doğrulanır; komut ve sequence geçerliyse DATA içeriği BRAM'e yazılır. BEGIN sırasında PicoRV32 reset altında tutulur.

END toplam byte sayısı ve transfer CRC32'sini doğrular; RUN ise işlemci resetini bırakarak mevcut FPGA hedefinde yürütmeyi 0x00000000 adresinden başlatır.



Şekil 2.5: FPGA loader durum makinesinin sadeleştirilmiş görünümü



Şekil 2.6: PicoRV32 bellek ve MMIO haritası

Tang Nano 9K üzerinde 27MHz giriş saati kullanılmaktadır. FPGA entegrasyonu, Gowin cihaz belgeleri ve açık kaynak UART çekirdeği temel alınarak gerçekleştirilmiştir [14]. UART haberleşme yapısı açık kaynak UART çekirdeğinden yararlanılarak geliştirilmiştir [13]. UART TX/RX pinleri sırasıyla 17 ve 18'dir. Altı aktif-düşük LED 10, 11, 13, 14, 15 ve 16 numaralı pinlere; aktif-düşük S1 ve S2 butonları 4 ve 3 numaralı pinlere bağlıdır. Buton girişleri senkronize edilerek yazılıma basılı durumda mantıksal 1 olarak sunulur.

2.7 Tkinter IDE İş Akışı

Tkinter tabanlı IDE, aynı anda birden fazla .asm dosyasını sekmelerde açabilmekte, her kaynağı ayrı object dosyasına assemble edebilmekte ve seçilen object dosyalarını linker script ile bağlayabilmektedir. UART loader penceresi COM portu ve .picoimg seçimi, bağlantı testi, yükleme ve çalıştırma işlemlerini sunar. Yükleme arka plan thread'inde yürütüldüğünden kullanıcı arayüzü uzun seri port işlemlerinde donmaz.

Bölüm 3

Deneyisel Çalışmalar, Test ve Analiz

3.1 Deney Tasarımı ve Test Senaryoları

Döngü, alt program, çoklu object, relocation, veri section'ları, MMIO ve bellek sınırlarını farklı düzeylerde zorlayan dört sistematik senaryo hazırlanmıştır. Her senaryo aynı FPGA bitstream'i üzerinde assemble, link, UART yükleme ve fiziksel davranış gözlemi adımlarından geçirilmiştir. Deneylerin tasarlanması, uygulanması ve sonuçlarının yorumlanması bu program çıktısıyla doğrudan ilişkilidir (PÇ7).

Tablo 3.1: Test senaryolarının kapsamı

Senaryo	Temel amaç	Beklenen kart davranışı	Doğrulanabilirleşenler
Walking-One Pattern	Döngü ve LED veri yolu	Tek bir LED sırayla ilerler	Assembler, JAL, MMIO, delay
Up/Down Counter	Buton kontrollü sayaç	S1 artırır, S2 azaltır; 0-63 sarar	Veri section'ları, buton/LED MMIO
Multi-Object Call/Relocation	Object'ler arası çağrı	Butona göre toplama/çıkarma sonucu gösterilir	Global/extern, JAL/JALR, relocation
Memory Boundary/Out-of-Range	BRAM sınır davranışı	Son geçerli ve ilk geçersiz adres ayrıştırılır	Linker, adres kontrolü, bellek arayüzü

Testlerin başarı ölçütleri fiziksel davranış ve araç zinciri çıktıları birlikte değerlendirilerek belirlenmiştir. Walking-One testinde aynı anda yalnızca bir LED'in düzenli olarak ilerlemesi; Up/Down Counter testinde S1 ile artış, S2 ile azalış ve 0-63 aralığında sarma; Multi-Object testinde buton seçimine göre doğru toplama veya çıkarma sonucunun LED'lerde görülmesi başarı kabul edilir. Memory Boundary testinde son geçerli adres ile ilk geçersiz adresin ayrıştırılması ve beklenen gösterge davranışının oluşması aranır. Ayrıca tüm olumlu senaryolarda assembler ve linker işlemlerinin hatasız tamamlanması, .picoimg doğrulamasının geçmesi ve UART yüklemesinin

ACK yanıtlarıyla tamamlanması gerekir.

3.1.1 Walking-One Pattern Test

Walking-one deseni, veri yolunun her bitinin tek başına etkinleştirilerek gözlenmesine dayanan klasik bir doğrulama yaklaşımıdır [15]. Senaryo, altı LED üzerinde tek bir mantıksal 1 değerini kaydırır. Döngüler, immediate işlemleri, dallanma, MMIO yazma ve ayrı delay alt programı birlikte sınanır. Kartta aynı anda tek LED'in sırayla hareket etmesi beklenir.

Adlandırma dayanağı: "Walking-One" adı, bellek ve veri yolu testlerinde her çevrimde yalnızca tek bitin etkinleştirildiği walking-ones yaklaşımından alınmıştır [15].

3.1.2 Up/Down Counter Test

Up/down counter, sayacın iki yönde güncellenmesini ve sınırdan sarma davranışını doğrular [16]. S1 butonu sayacı artırır, S2 azaltır; düşük altı bit LED'lerde gösterilir. Başlangıç değeri sıfırdır ve değer 0-63 arasında modüller olarak sarar. Senaryo, buton MMIO okumasını, LED MMIO yazmasını, .data/.bss yerleşimini ve veri sembollerine ilişkin relocation işlemlerini doğrular.

Adlandırma dayanağı: "Up/Down Counter" terimi, sayısal tasarım literatüründe artan ve azalan yönde sayabilen sayaç devrelerini tanımlar [16].

3.1.3 Multi-Object Call/Relocation Test

Bu senaryoda ana program ile toplama/çıkarma alt programları ayrı object dosyalarında tutulur. .global ve .extern sembolleri, object'ler arası jal/jalr çağrılar ve HI20/LO12 relocation işlemleri doğrulanır. S1 ve S2 butonlarına göre farklı alt programların çalışması, sembol çözümlemenin fiziksel kart davranışıyla gözlenmesini sağlar.

Adlandırma dayanağı: Senaryo adı, birden fazla object dosyası arasında sembol çözümleme ve RISC-V relocation kayıtlarının uygulanmasını doğrudan ifade eder [2].

3.1.4 Memory Boundary/Out-of-Range Test

Sınır değer analizi, aralıkların hemen içindeki ve dışındaki değerlerin seçilmesini önerir [17]. Bu senaryo 16 KiB BRAM'in son geçerli word adresi 0x00003FFC ile ilk geçersiz adresi 0x00004000 çevresindeki davranışı sınar. Amaç, linker ve donanım bellek arayüzünün sınır koşullarında beklenen denetimleri uyguladığını göstermektir. Geçersiz adrese yazmanın genel amaçlı bir güvenlik mekanizması olmadığı, yalnızca bu sistemin adres aralığı kontrolünü sınıadığı özellikle belirtilmelidir [18].

Adlandırma dayanağı: "Memory Boundary/Out-of-Range" adı, sınır değer analizi ile aralık dışı bellek erişimi sınıflandırmalarını birlikte yansıtır

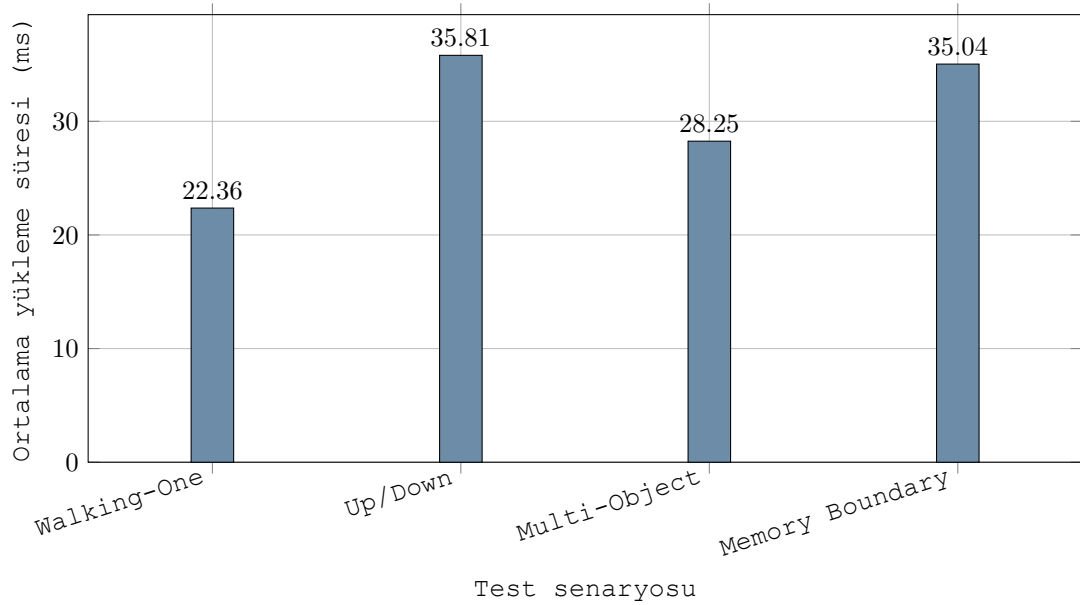
[17]. Geçersiz bellek erişiminin güvenlik sınıflandırması ayrıca CWE-787 tanımında açıklanmaktadır [18].

3.2 Veri Toplama ve Donanım Metrikleri

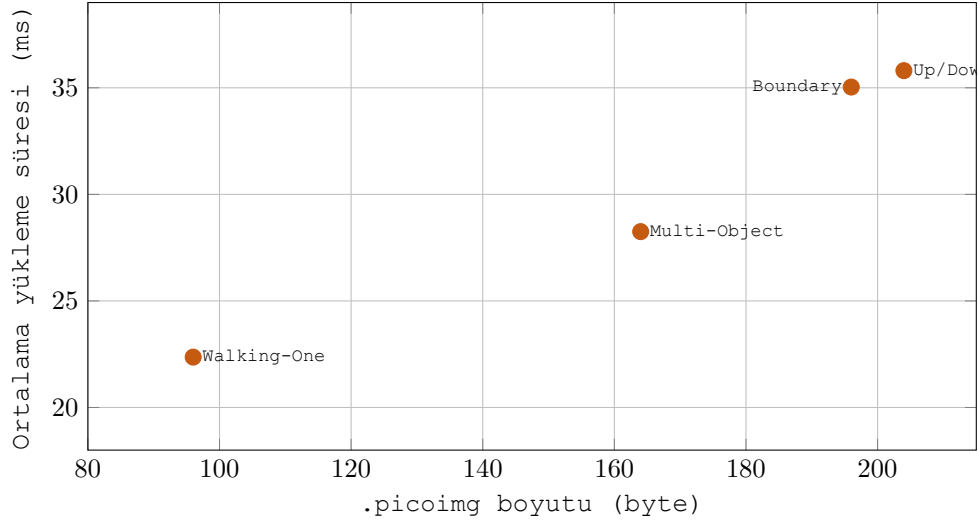
Farklı kod boyutları için gerçek UART yükleme süreleri ve Gowin PNR donanım kaynak tüketimleri toplanmıştır. UART deneyleri COM7 üzerinde 115.200baud hızında, her senaryo için on tekrar ile yapılmıştır. Ölçüm aralığı PING'i dışarıda bırakmakta; BEGIN, DATA, END, RUN ve ilgili ACK yanıtlarını kapsamaktadır.

Tablo 3.2: Senaryo bazlı program ve UART yükleme metrikleri

Senaryo	Kaynak (B)	Program (B)	Image (B)	Paket	Ort. (ms)	Min. (ms)	Maks. (ms)	σ (ms)
Walking-One Pattern	674	48	96	4	22,364	22,085	22,698	0,152
Up/Down Counter	2304	156	204	5	35,805	35,593	36,085	0,122
Multi-Object Call/Relocation	2074	116	164	4	28,251	28,103	28,417	0,086
Memory Boundary/Out-of-Range	2376	148	196	5	35,035	34,843	35,266	0,113



Şekil 3.1: CSV verisinden üretilen ortalama UART yükleme süreleri



Şekil 3.2: .picoimg boyutu ile ortalama yükleme süresi ilişkisi

3.2.1 Yükleme Süresi Sonuçlarının Analizi

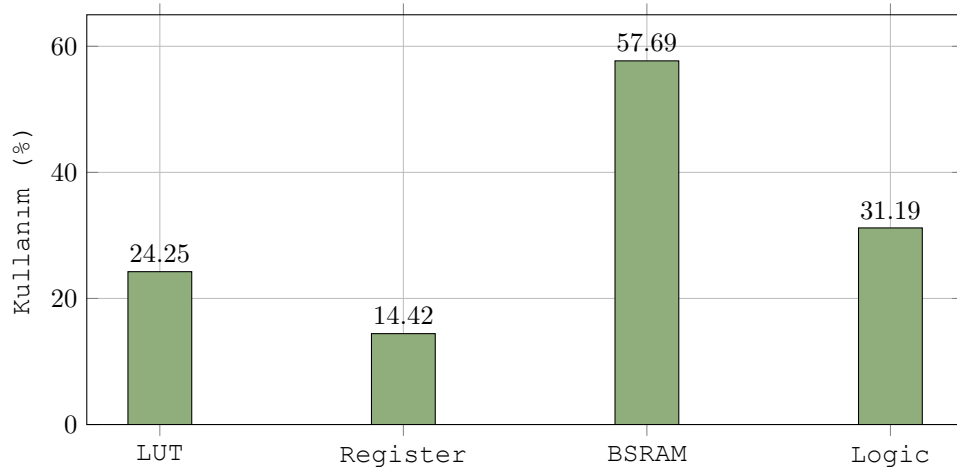
Ölçümler, yüklenen veri ve paket sayısı arttıkça yükleme süresinin genel olarak arttığını göstermektedir. En küçük .picoimg olan Walking-One (96byte) dört UART paketiyle 22,364ms ortalamaya sahiptir. En büyük image olan Up/Down Counter (204byte) beş paketle 35,805ms ortalamaya ulaşmıştır. İlişki yalnızca byte sayısı ile doğrusal değildir; her ek paket başlık, CRC32, UART iletim süresi, FPGA işleme süresi ve ACK bekleme maliyeti getirir. Dört paketli Multi-Object testinin, benzer boyuttaki beş paketli sınır testinden daha kısa sürmesi bu paketleme etkisiyle uyumludur.

Standart sapmaların 0,086ms ile 0,152ms aralığında kalması, aynı koşullarda tekrarlanan ölçümlerin kararlı olduğunu göstermektedir. Bununla birlikte sonuçlar yalnızca COM7, kullanılan bilgisayar, işletim sistemi zamanlaması ve mevcut USB-UART bağlantısı için geçerlidir; farklı host sistemlerde yeniden ölçülmelidir.

3.2.2 FPGA Kaynak Tüketimi ve Zamanlama

Tablo 3.3: Gowin PNR donanım kaynakları

Kaynak	Kullanılan	Toplam	Kullanım (%)
LUT	2095	8640	24,25
Register	965	6693	14,42
BSRAM	15	26	57,69
Toplam Logic	2695	8640	31,19



Şekil 3.3: FPGA kaynak kullanım yüzdeleri

PNR raporunda azami frekans 42,442MHz olarak belirlenmiştir; sistem 27MHz saatle çalıştığından pozitif zamanlama payı bulunmaktadır. BSRAM'in %57,69 ile en yüksek oranlı kaynak olması, 16 KiB program/veri belleği ve gerekli bellek yapılarının blok RAM üzerinde gerçekleştirilmesinden kaynaklanmaktadır.

LUT ve register kullanımı yalnızca PicoRV32 çekirdeğinden değil; UART alıcı/verici mantığı, CRC32 hesaplama birimi, paket ayrıştırma, loader FSM, ACK/NACK yanıt üretimi ve MMIO adres çözümleme mantığından da kaynaklanmaktadır. Bu nedenle kaynak tüketimi tek bir test programına değil, ortak UART-loader bitstream'inin tamamına aittir.

Farklı Assembly senaryolarının LUT, register ve BSRAM değerleri değişmez. Bunun nedeni programların sentezlenerek bitstream'e eklenmemesi; çalışma zamanında UART üzerinden aynı BRAM'e yüklenmesidir. Dolayısıyla donanım kaynak tablosu her senaryonun değil, ortak PicoRV32 UART-loader bitstream'inin maliyetini göstermektedir.

Bölüm 4

Projenin Küresel, Toplumsal ve Ekonomik Etkileri

4.1 Sürdürülebilirlik ve Yeşil Bilişim

Bu projede kullanılan PicoRV32 çekirdeği, alan verimliliği gözetilerek tasarlanmış küçük bir soft-core işlemcidir. Küçük çekirdek ve yalnızca gerekli çevre birimlerinin kullanılması, daha büyük ve kullanılmayan özellikler içeren tasarımlara kıyasla daha az FPGA kaynağı kullanma potansiyeli taşır. Bununla birlikte RISC-V komut kümesinin veya geliştirilen loader'ın tek başına doğrudan enerji tasarrufu sağladığı iddia edilemez; enerji etkisinin kanıtlanması için güç analizleri ve karşılaştırmalı ölçümler gerekir.

Yazılım boyutunun küçültülmesi ve gereksiz UART paketlerinin azaltılması, aktarım süresini ve aktif çalışma süresini azaltabilir. Aynı donanım üzerinde yeni programların yüklenebilmesi, eğitim ve prototipleme süreçlerinde yeni donanım üretme ihtiyacını sınırlar. Bu yönleriyle çalışma, BM Sürdürülebilir Kalkınma Amaçlarından SKA 7 "Erişilebilir ve Temiz Enerji" ve SKA 13 "İklim Eylemi" ile dolaylı olarak ilişkilendirilebilir (PÇ8).

4.2 Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık

RISC-V ISA, PicoRV32 ve kullanılan UART çekirdeğinin açık kaynaklı olması; lisans maliyeti olmadan mimarinin incelenmesine, değiştirilmesine ve yerel ihtiyaçlara göre genişletilmesine olanak tanır. Bu projede assembler, linker, loader image biçimi, host yazılımı ve FPGA entegrasyonunun özgün olarak geliştirilmesi, hazır kapalı araçlara bağımlılığını azaltan bir bilgi birikimi oluşturmuştur.

Açık bir araç zinciri özellikle eğitim, erken aşama Ar-Ge ve yerli çip ekosistemi açısından erişim engellerini azaltabilir. Buna karşılık doğrulama, bakım, standart uyumluluğu ve uzman iş gücü maliyetleri ortadan kalkmaz. Bu nedenle ekonomik katkı, doğrudan maliyet kazancı olarak değil; geliştirilebilir teknoloji yetkinliği ve dışa bağımlılığını azaltma potansiyeli olarak değerlendirilmiştir. Bu yaklaşım SKA 8 "İnsana Yakışır İş ve Ekonomik Bū-

yüme" ile SKA 9 "Sanayi, Yenilikçilik ve Altyapı" hedefleriyle ilişkilidir (PÇ8).

4.3 Fonksiyonel Güvenlik ve Sağlık

Güvenilirlik tek bir CRC kontrolüne bırakılmamıştır. Object/linker düzeyindeki adres ve taşma kontrolleri, .picoimg segment ve image CRC32 değerleri, UART paket CRC32 değeri, sequence takibi, ACK/NACK ve retry mekanizması birbirini tamamlayan savunma katmanlarıdır. CPU'nun aktarım süresince reset altında tutulması, kısmen yüklenmiş programın çalışmasını engeller. Bu ilkeler; tıbbi cihaz, otomotiv ve savunma sistemlerinde hatalı yazılım görünümlerinin çalıştırılmasını önleme açısından önemlidir ve SKA 3 "Sağlık ve Kaliteli Yaşam" ile ilişkilendirilebilir (PÇ8).

Ancak geliştirilen prototip sertifikalı bir fonksiyonel güvenlik çözümü değildir. Kritik sistemlerde kullanımdan önce tehdit modellemesi, kimlik doğrulama, güvenli boot, yedekleme, hata enjeksiyonu testleri ve ilgili sektör standartlarına uygun doğrulama gereklidir.

4.4 E-Atık Yönetimi ve Döngüsel Ekonomi

FPGA tabanlı sistemlerin yeniden yapılandırılabilir olması ve kullanıcı programlarının loader üzerinden güncellenebilmesi, işlev değişikliği veya hata düzeltilmesi için donanımın tamamen değiştirilmesi gereksinimini azaltabilir. Bu durum cihaz ömrünün uzatılmasına ve elektronik atığın azaltılmasına katkı sağlama potansiyeline sahiptir; dolayısıyla SKA 12 "Sorumlu Üretim ve Tüketim" ile ilişkilidir (PÇ8).

Bu projedeki UART yükleme işlemi yerel USB-UART bağlantısıyla yapılmaktadır ve henüz ağ üzerinden gerçek bir OTA mekanizması değildir. Buna rağmen loader mimarisi, gelecekte kimlik doğrulamalı uzaktan güncelleme katmanının eklenebileceği temel bir yeniden programlama altyapısı sunmaktadır.

Bölüm 5

Proje Yönetimi ve Takım Çalışması

5.1 Görev Dağılımı ve Sorumluluk Matrisi

Proje, Oğuzhan Özen ve Geydanur Tuba Yılmaz tarafından birlikte yürütülmüştür. Teknik modüller, deneyler ve raporlama görevleri iki öğrenci arasında paylaşılmış; entegrasyon kararları ortak değerlendirilmiştir. Bu çalışma biçimi bireysel sorumluluk alma ve disiplin içi takım çalışması becerilerini desteklemektedir (PÇ12, PÇ13). Ders sorumlusu, gereksinimlerin yorumlanması ve akademik değerlendirme açısından danışılan ve bilgilendirilen paydaştır. Tablo 5.1, proje kapsamındaki sorumluluk dağılımını göstermektedir.

Tablo 5.1: Proje RACI matrisi

İş paketi	Öğrenci 1	Öğrenci 2	Ders Sorumlusu
Gereksinim ve literatür analizi	R/A	R	I
Assembler ve object biçimi	R	R/A	I
Linker script ve relocation sistemi	R	R/A	I
UART protokolü ve Python host loader	R	R/A	I
FPGA loader, PicoRV32 ve MMIO entegrasyonu	R/A	R	I
Test senaryoları, ölçüm ve analiz	R/A	R	I
Raporlama ve sürüm yönetimi	R	R/A	I

Rol karşılıkları: Öğrenci 1: Geydanur Tuba Yılmaz; Öğrenci 2: Oğuzhan Özen; Ders Sorumlusu: Prof. Dr. Halit Öztekin. R: Uygulayıcı (Responsible), A: Nihai sorumlu (Accountable), C: Danışılan (Consulted), I: Bilgilendirilen (Informed).

5.2 Koordinasyon ve Sürüm Kontrol Yönetimi

Geliştirme süreci modül bazlı ve artımlı olarak yürütülmüştür. Önce assembler/linker altyapısı, ardından .picoimg biçimi ve host loader, son olarak FPGA loader ve fiziksel kart testleri tamamlanmıştır. Entegrasyon aşamala-

rında önce yazılım düzeyinde doğrulama, ardından Gowin sentez/PNR ve gerçek UART yükleme deneyi uygulanmıştır.

Kaynak kod ve belgeler Git ile sürümlenmiş, GitHub deposuna anlamlı commit'ler hâlinde aktarılmıştır. Deney çıktıları ve ham UART ölçümleri de depoda saklanarak sonuçların izlenebilirliği sağlanmıştır. İki öğrenci arasındaki koordinasyon, modül sorumluluklarının belirlenmesi, ortak entegrasyon kontrolleri ve sürüm geçmişinin izlenmesi üzerinden yürütülmüştür. Danışman geri bildirimleri ve gereksinim kontrolleri de koordinasyon noktaları olarak kullanılmıştır (PÇ13).

Bölüm 6

Bireysel Katkı Beyanı

Bu bölüm her öğrenci tarafından ayrı ayrı tamamlanmalı ve imzalanmalıdır (PÇ12).

GEYDANUR TUBA YILMAZ

Grup çalışmasından bağımsız olarak tamamen kendi başıma tasarladığım ve gerçekleştirdiğim modüller:

FPGA tarafında PicoRV32 işlemci çekirdeğinin kurulması ve sistemle uyumlu hâle getirilmesi üzerinde çalıştım. İşlemci çekirdeğinin BRAM, loader yapısı ve LED/buton MMIO birimleri ile bağlantısını kurdum. İlk aşamada, projenin temel çalışma mantığını test edebilmek için işimizi görecektir şekilde küçük bir işlemci çekirdeği yazdım. Daha sonra hazır PicoRV32 işlemci çekirdeğini sisteme entegre ederek programların FPGA üzerinde çalışmasını sağladım.

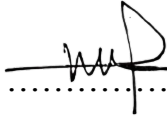
Tek başıma çözdüğüm en büyük teknik problem ve çözüm yaklaşımım:

İlk karşılaştığım sorun, hazır PicoRV32 çekirdeğinin projedeki bellek ve loader yapısıyla doğrudan uyumlu çalışmamasıydı. Bu nedenle önce basit bir işlemci çekirdeğiyle sistemin temel akışını denedim. Ardından PicoRV32'nin bellek bağlantılarını ve reset kontrolünü projeye uygun şekilde düzenleyerek işlemcinin loader tamamlandıktan sonra doğru şekilde çalışmasını sağladım.

Ad Soyad: GEYDANUR TUBA YILMAZ

Öğrenci Numarası: 23010903086

Tarih: 7 Haziran 2026

İmza: 

OĞUZHAN ÖZEN

Grup çalışmasından bağımsız olarak tamamen kendi başıma tasarladığım ve gerçekleştirdiğim modüller:

UART protokolü ve Python host loader kısmı üzerinde çalıştım. .picoimg dosyasının bilgisayar tarafında okunması, paketlere ayrılması ve UART üzerinden FPGA'ya gönderilmesi işlemlerini gerçekleştirdim. Başlangıçta loader işlemleri bash/komut satırı üzerinden yürütülüyordu. Daha sonra bu işlemleri tasarlanan assembler IDE'sine entegre ederek yükleme sürecinin arayüz üzerinden yapılmasını sağladım.

Tek başıma çözdüğüm en büyük teknik problem ve çözüm yaklaşımım:

Karşılaştığım en büyük sorun, UART yükleme işleminin komut satırı üzerinden yürütülmesinin kullanım açısından zor ve hataya açık olmasıydı. Bu sorunu, host loader akışını IDE içine taşıyarak çözdüm. Böylece kullanıcı assemble, link ve UART ile yükleme işlemlerini ayrı ayrı komut yazmadan aynı arayüz üzerinden gerçekleştirebilir hâle geldi.

Ad Soyad: OĞUZHAN ÖZEN

Öğrenci Numarası: 22010903114

Tarih: 7 Haziran 2026

İmza:


Bölüm 7

Sonuç ve Gelecek Çalışmalar

Bu projede Assembly kaynağından fiziksel FPGA davranışına uzanan tam bir sistem programlama zinciri gerçekleştirilmiştir. Adres-bağımsız object üretimi, linker-script tabanlı yerleştirme, relocation çözümü, tek dosyalı adresli image, CRC32 korumalı UART protokolü ve FPGA loader FSM aynı sistemde birleştirilmiştir. Dört test senaryosu en az üç farklı karmaşıklık düzeyi gereksinimini aşarak döngü, alt program, çoklu object, relocation, MMIO ve bellek sınırlarını doğrulamıştır. Gerçek kart ölçümleri yükleme sürelerini, PNR sonuçları ise donanım maliyetini nicel olarak ortaya koymuştur.

Gelecek çalışmalarda özel JSON object biçimi yerine veya yanında ELF desteği eklenebilir; SPI/flash tabanlı kalıcı boot zinciri geliştirilebilir; paketlere kimlik doğrulama ve isteğe bağlı sıkıştırma uygulanabilir; entry adresi ve çoklu bellek bölgeleri donanımda genelleştirilebilir. Donanım-içinde-test mekanizmaları ve CI ortamında assembler/linker regresyon testleri ile uzun süreli bakım güvenilirliği artırılabilir. Daha yüksek baud hızları ve farklı payload boyutları deneysel olarak karşılaştırılarak aktarım verimliliği optimize edilebilir.

7.1 Sınırlamalar

- Program ve veriler toplam 16 KiB BRAM ile sınırlıdır.
- .picoimg formatı entry bilgisi taşısa da mevcut FPGA hedefinde işlemci reset sonrasında 0x00000000 adresinden başlatılmaktadır.
- UART hızı 115.200baud ve DATA payload üst sınırı 128 byte'tır.
- Object ve .picoimg biçimleri projeye özeldir; standart ELF desteği yoktur.
- Loader bitstream'i SRAM'e Gowin Programmer ile yüklenir; güç kesildiğinde kalıcı boot yoktur.
- CRC32 kasıtsız iletim hatalarını tespit eder, kötü niyetli değişikliğe karşı kimlik doğrulama sağlamaz.
- Deneyler tek kart, tek host ve tek seri port yapılandırmasında yürütülmüştür.

Bölüm 8

Kaynakça

- [1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, resmî ISA belirtimi, erişim: 2026.
- [2] RISC-V International, *RISC-V ABIs Specification*, relocation ve çağrı kuralları, resmî ABI belirtimi, erişim: 2026.
- [3] C. Wolf, "PicoRV32: A Size-Optimized RISC-V CPU," açık kaynak yazılım/donanım deposu, <https://github.com/YosysHQ/picorv32>, erişim: 2026.
- [4] M. Barr ve A. Massa, *Programming Embedded Systems*, kitap, 2. bs., O'Reilly Media, 2006.
- [5] J. W. Valvano, *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers*, kitap, CreateSpace, 2014.
- [6] D. Black, J. Donovan, B. Bunton ve A. Keist, *SystemC: From the Ground Up*, kitap, Springer, 2010.
- [7] IEEE, *IEEE Standard for Test Access Port and Boundary-Scan Architecture*, standart, IEEE Std 1149.1, erişim: 2026.
- [8] Texas Instruments, *JTAG Interface Reference and Programming Guide*, teknik başvuru dokümanı, erişim: 2026.
- [9] W. Stallings, *Data and Computer Communications*, kitap, Pearson, erişim: 2026.
- [10] J. Postel, *Internet Protocol*, standart belirtimi, RFC 791, 1981.
- [11] W. W. Peterson ve D. T. Brown, "Cyclic Codes for Error Detection," *Proceedings of the IRE*, akademik makale, c. 49, no. 1, ss. 228-235, 1961.
- [12] A. S. Tanenbaum ve D. J. Wetherall, *Computer Networks*, kitap, 5. bs., Pearson, 2011.
- [13] A. Forencich, "Verilog UART Core," açık kaynak donanım modülü, MIT Lisansı, erişim: 2026.
- [14] Gowin Semiconductor, *Gowin FPGA Designer User Guide ve Tang Nano 9K Cihaz Belgeleri*, üretici teknik dokümantasyonu, erişim: 2026.

- [15] AMD/Xilinx, *Xilinx Memory Test Application: Walking Ones Test*, üretici teknik dokümantasyonu, erişim: 2026.
- [16] AMD/Xilinx, *Binary Counter / Up-Down Counter IP Documentation*, üretici teknik dokümantasyonu, erişim: 2026.
- [17] ISTQB, *Certified Tester Foundation Level Syllabus: Boundary Value Analysis*, resmî eğitim/dokümantasyon kaynağı, erişim: 2026.
- [18] MITRE, "CWE-787: Out-of-bounds Write," Common Weakness Enumeration, <https://cwe.mitre.org/data/definitions/787.html>, erişim: 2026.

Kısa video:

<https://drive.google.com/file/d/1qoOTWrEMSWeCFxiWzbByl97agABoIrZS/view?usp=sharing>

Proje Github Reposu:

<https://github.com/oguzhanozen/assembler>

Bölüm A

Program Öğrenme Çıktıları

Tablo A.1: Program Öğrenme Çıktılarının tam listesi

No	Program Öğrenme Çıktısı
1	Matematik, fen bilimleri, temel mühendislik, bilgisayarla hesaplama ve ilgili mühendislik disiplinine özgü konularda bilgi.
2	Matematik, fen bilimleri, temel mühendislik, bilgisayarla hesaplama ve ilgili mühendislik disiplinine özgü bilgileri, karmaşık mühendislik problemlerinin çözümünde kullanabilme becerisi.
3	Karmaşık mühendislik problemlerini, temel bilim, matematik ve mühendislik bilgilerini kullanarak ve ele alınan problemle ilgili BM Sürdürülebilir Kalkınma Amaçlarını gözeterek tanımlama, formüle etme ve analiz becerisi.
4	Karmaşık mühendislik problemlerine yaratıcı çözümler tasarlama becerisi; karmaşık sistemleri, süreçleri, cihazları veya ürünleri gerçekçi kısıtları ve koşulları gözeterek, mevcut ve gelecekteki gereksinimleri karşılayacak biçimde tasarlama becerisi.
5	Karmaşık mühendislik problemlerinin analizi ve çözümüne yönelik, tahmin ve modelleme de dahil olmak üzere, uygun teknikleri, kaynakları ve modern mühendislik ve bilişim araçlarını, sınırlamalarının da farkında olarak seçme ve kullanma becerisi.
6	Karmaşık mühendislik problemlerinin incelenmesi için literatür araştırması ve araştırma yöntemlerini kullanma becerisi.
7	Karmaşık mühendislik problemlerinin incelenmesi için deney tasarlama, deney yapma, veri toplama, sonuçları analiz etme ve yorumlama becerisi.
8	Mühendislik uygulamalarının BM Sürdürülebilir Kalkınma Amaçları kapsamında, topluma, sağlık ve güvenliğe, ekonomiye, sürdürülebilirlik ve çevreye etkileri hakkında bilgi.
9	Mühendislik çözümlerinin hukuksal sonuçları konusunda farkındalık.
10	Mühendislik meslek ilkelerine uygun davranma ve etik sorumluluk hakkında bilgi.
11	Hiçbir konuda ayrımcılık yapmadan, tarafsız davranma ve çeşitliliği kapsayıcı olma konularında farkındalık.
12	Bireysel olarak etkin biçimde çalışabilme becerisi.
13	Disiplin içi takımlarda (yüz yüze, uzaktan veya karma) takım üyesi veya lideri olarak etkin biçimde çalışabilme becerisi.
14	Çok disiplinli takımlarda (yüz yüze, uzaktan veya karma) takım üyesi veya lideri olarak etkin biçimde çalışabilme becerisi.

No	Program Öğrenme Çıktısı
15	Hedef kitlenin çeşitli farklılıklarını (eğitim, dil, meslek gibi) dikkate alarak, teknik konularda sözlü, yazılı etkin iletişim kurma becerisi.
16	Proje yönetimi ve ekonomik yapılabilirlik analizi gibi iş hayatındaki uygulamalar hakkında bilgi; girişimcilik ve yenilikçilik hakkında farkındalık.
17	Bağımsız ve sürekli öğrenebilme, yeni ve gelişmekte olan teknolojilere uyum sağlayabilme ve teknolojik değişimlerle ilgili sorgulayıcı düşünebilmeyi kapsayan yaşam boyu öğrenme becerisi.

Bölüm B

UART Komutları ve Hata Kodları

Tablo B.1: UART protokol komutları

Kod	Komut	İşlev
0x01	PING	Bağlantıyı ve sequence eşzamanlamasını doğrular.
0x02	BEGIN	CPU'yu reset altında tutar ve aktarımı başlatır.
0x03	DATA	Payload byte'larını paket adresine yazar.
0x04	END	Toplam byte ve transfer CRC32 değerini doğrular.
0x05	RUN	CPU resetini bırakır ve programı çalıştırır.
0x80	ACK	Komutun kabul edildiğini bildirir.
0x81	NACK	Komutun hata koduyla reddedildiğini bildirir.

Tablo B.2: UART protokol hata durumları

Kod	Durum	Açıklama
0	OK	İşlem başarılıdır.
1	BAD_CRC	Paket CRC32 doğrulaması başarısızdır.
2	BAD_SEQUENCE	Beklenmeyen sequence değeri alınmıştır.
3	BAD_COMMAND	Komut türü desteklenmemektedir.
4	BAD_ADDRESS	Yazma veya çalıştırma adresi geçersizdir.
5	BAD_LENGTH	Payload veya toplam aktarım uzunluğu geçersizdir.
6	NOT_READY	Komut mevcut loader durumunda uygulanamaz.
7	HOST_INTERNAL	Host tarafında iç hata oluşmuştur.

Bölüm C

Tekrar Üretim ve Test Kodları

Kod Bloğu C.1: Temel komut satırı iş akışı

```
1 # Start the IDE
2 python main.py
3
4 # Link object files with the linker script
5 python -m src.linker outputs/ assembler/main.o outputs/ assembler/delay.o \
6     -T tests/project.ld -o outputs/linker/program
7
8 # Validate image structure and CRC values
9 python -m src.loader_image outputs/loader/program.picoimg
10
11 # Validate packet generation without a physical connection
12 python -m src.host_loader outputs/loader/program.picoimg --dry-run
13
14 # Load the program on the physical board
15 python -m src.host_loader outputs/loader/program.picoimg --port COM7
16
17 # Collect metrics over ten trials
18 python metrics/collect_metrics.py --port COM7 --baud 115200 --trials 10
```

Kartın gücü kesilmişse önce Gowin Programmer ile outputs/fpga/picorv_loader.fs bitstream'i SRAM'e yüklenmelidir. Güç kesilmediği sürece farklı .picoimg programları Gowin Programmer tekrar kullanılmadan UART üzerinden yüklenebilir.

C.1 Walking-One Pattern Test Kodları

Kod Bloğu C.2: Walking-One ana programı

```
1 .text
2
3 .global _start
4 .global loop
5
6 .extern delay_start
7
```



```

8 _start:
9     lui    x5, 0x10000          # x5 = 0x10000000, LED address
10    addi   x6, x0, 1            # x6 = LED pattern
11
12 loop:
13    sw     x6, 0(x5)            # Write pattern to LEDs
14
15    lui    x7, 0x01000          # Initialize delay counter
16    jal    x0, delay_start      # delay_start is defined in another object

```

Kod Bloğu C.3: Walking-One gecikme alt programı

```

1 .text
2
3 .global delay_start
4
5 .extern loop
6
7 delay_start:
8     addi   x7, x7, -1
9     bne    x7, x0, delay_start
10
11    slli    x6, x6, 1           # Shift the LED pattern left
12    addi    x8, x0, 64          # Restart after all six LEDs are visited
13    bne     x6, x8, loop        # loop is defined in another object
14
15    addi    x6, x0, 1
16    jal     x0, loop            # Return to the external loop symbol

```

C.2 Up/Down Counter Test Kodları

Kod Bloğu C.4: Up/Down Counter ana programı

```

1 .init
2 .text
3 .global _start
4 .extern led_reg          # External symbol containing the LED address
5 .extern s1_reg           # External symbol containing the S1 address
6 .extern s2_reg           # External symbol containing the S2 address
7 .extern counter_bss      # Counter allocated in an external .bss section
8
9 _start:
10    lui    x5, %hi(led_reg)
11    lw     x6, %lo(led_reg)(x5)    # x6 = LED address
12    lui    x5, %hi(s1_reg)
13    lw     x7, %lo(s1_reg)(x5)    # x7 = S1 address
14    lui    x5, %hi(s2_reg)
15    lw     x8, %lo(s2_reg)(x5)    # x8 = S2 address
16

```

```

17  # Resolve and clear the counter allocated in the external .bss section
18  lui x5, %hi(counter_bss)
19  addi x9, x5, %lo(counter_bss)    # x9 = counter RAM address
20  addi x10, x0, 0
21  sw x10, 0(x9)                    # counter_bss = 0
22
23  input_wait:
24      lw x11, 0(x7)                  # Read S1
25      andi x11, x11, 1
26      bne x11, x0, increment_count
27
28      lw x12, 0(x8)                  # Read S2
29      andi x12, x12, 2
30      bne x12, x0, decrement_count
31
32      jal x0, input_wait
33
34  increment_count:
35      lw x13, 0(x9)                  # Read counter from RAM
36      addi x13, x13, 1                # i++
37      addi x14, x0, 64                # Check upper boundary
38      blt x13, x14, save_and_show
39      addi x13, x0, 0
40      jal x0, save_and_show
41
42  decrement_count:
43      lw x13, 0(x9)                  # Read counter from RAM
44      addi x13, x13, -1               # i--
45      bge x13, x0, save_and_show     # Check lower boundary
46      addi x13, x0, 63                # Wrap to maximum value
47
48  save_and_show:
49      sw x13, 0(x9)                  # Store updated counter in RAM
50      sw x13, 0(x6)                  # Display counter as an LED bit pattern
51
52  debounce_wait:
53      lw x11, 0(x7)
54      andi x11, x11, 1
55      lw x12, 0(x8)
56      andi x12, x12, 2
57      or x14, x11, x12
58      bne x14, x0, debounce_wait    # Wait until both buttons are released
59      jal x0, input_wait
60  .end

```

Kod Bloğu C.5: Up/Down Counter bellek haritası sembolleri

```

1  .data
2  .align 4
3  .global led_reg
4  .global s1_reg

```

```

5 .global s2_reg
6
7 led_reg:      .word 0x10000000
8 s1_reg:       .word 0x10000004
9 s2_reg:       .word 0x10000004
10
11 .section .bss, "aw", @nobits          # Linker-script NOBITS parsing test
12 .align 4
13 .global counter_bss
14 counter_bss:
15     .space 4                          # Allocates 4 bytes in RAM but not in the HEX
16     image
17 .end

```

C.3 Multi-Object Call/Relocation Test Kodları

Kod Bloğu C.6: Multi-Object ana programı

```

1 .init
2 .text
3 .global _start
4 .extern fonksiyon_topla    # Function is defined in another object
5 .extern fonksiyon_cikar   # Linker resolves this external function
6
7 _start:
8     # Resolve addresses from .data by applying relocation records
9     lui x5, %hi(led_addr)
10    lw x6, %lo(led_addr)(x5)    # x6 = 0x10000000, LED address
11    lui x5, %hi(s1_addr)
12    lw x7, %lo(s1_addr)(x5)    # x7 = 0x10000004, button status register
13    lui x5, %hi(s2_addr)
14    lw x8, %lo(s2_addr)(x5)    # x8 = 0x10000004, same status register
15
16    # Constant input operands
17    addi x10, x0, 12            # A = 12
18    addi x11, x0, 4            # B = 4
19
20 main_loop:
21    lw x12, 0(x7)              # Read S1
22    andi x12, x12, 1
23    bne x12, x0, call_topla    # Call external add function when S1 is pressed
24
25    lw x13, 0(x8)              # Read S2
26    andi x13, x13, 2
27    bne x13, x0, call_cikar    # Call external subtract function when S2 is
                                # pressed
28
29    # Turn LEDs off when no button is pressed
30    addi x14, x0, 0

```

```

31     sw    x14, 0(x6)
32     jal   x0, main_loop
33
34 call_topla:
35     jal   x1, fonksiyon_topla      # Call function from another object (ra = x1)
36     jal   x0, ekrana_bas
37
38 call_cikar:
39     jal   x1, fonksiyon_cikar      # Call function from another object (ra = x1)
40
41 ekrana_bas:
42     sw    x14, 0(x6)              # Display result as an LED bit pattern
43     jal   x0, main_loop
44
45 .data
46 .align 4
47 led_addr:    .word 0x10000000
48 s1_addr:     .word 0x10000004
49 s2_addr:     .word 0x10000004
50 .end

```

Kod Bloğu C.7: Multi-Object matematik alt programları

```

1  .text
2  .global fonksiyon_topla    # Export add function to the linker
3  .global fonksiyon_cikar   # Export subtract function to the linker
4
5  fonksiyon_topla:
6      add   x14, x10, x11      # x14 = A + B
7      jalr  x0, 0(x1)         # Return to main1.asm through x1/ra
8
9  fonksiyon_cikar:
10     sub    x14, x10, x11      # x14 = A - B
11     jalr  x0, 0(x1)         # Return to the caller
12 .end

```

C.4 Memory Boundary/Out-of-Range Test Kodları

Kod Bloğu C.8: Memory Boundary ana programı

```

1  .init
2  .text
3  .global _start
4  .extern led_ptr          # External symbols resolved by the linker
5  .extern s1_ptr
6  .extern s2_ptr
7  .extern test_pattern     # Test pattern defined in another object
8
9  _start:

```

```

10  lui x5, %hi(led_ptr)
11  lw  x6, %lo(led_ptr)(x5)      # x6 = LED address
12  lui x5, %hi(s1_ptr)
13  lw  x7, %lo(s1_ptr)(x5)      # x7 = S1 address
14  lui x5, %hi(s2_ptr)
15  lw  x8, %lo(s2_ptr)(x5)      # x8 = S2 address
16
17  # Load external test data by applying a relocation record
18  lui x5, %hi(test_pattern)
19  lw  x10, %lo(test_pattern)(x5) # x10 = 0x000000AA
20
21  main_check:
22  lw  x11, 0(x7)                # Read S1
23  andi x11, x11, 1
24  bne x11, x0, safe_boundary_write # Write to the valid boundary when S1 is
    pressed
25
26  lw  x12, 0(x8)                # Read S2
27  andi x12, x12, 2
28  bne x12, x0, overflow_forced_write # Trigger out-of-range write when S2 is
    pressed
29
30  jal x0, main_check
31
32  safe_boundary_write:
33  # Last valid word address of the 16 KiB BRAM: 0x00003FFC
34  lui x13, 4                    # x13 = 0x00004000
35  addi x13, x13, -4             # x13 = 0x00003FFC, last valid address
36
37  sw  x10, 0(x13)               # Write inside the valid boundary
38
39  addi x14, x0, 1               # Success code: 0x01
40  sw  x14, 0(x6)                # Turn on LED0 to indicate success
41  jal x0, debounce_loop
42
43  overflow_forced_write:
44  # First address outside the 16 KiB BRAM: 0x00004000
45  lui x13, 1
46  slli x13, x13, 4              # x13 = 0x00004000, boundary violation
47
48  sw  x10, 0(x13)               # Force an out-of-range hardware write
49
50  addi x14, x0, 0x3F            # Error code: 0x3F, all LEDs
51  sw  x14, 0(x6)                # Turn on all LEDs to indicate an error
52
53  debounce_loop:
54  lw  x11, 0(x7)
55  andi x11, x11, 1
56  lw  x12, 0(x8)
57  andi x12, x12, 2

```

```
58     or    x14, x11, x12
59     bne   x14, x0, debounce_loop    # Wait until both buttons are released
60     jal   x0, main_check
61 .end
```

Kod Bloğu C.9: Memory Boundary güvenli veri section'ı

```
1  .data
2  .align 4
3  .global led_ptr
4  .global s1_ptr
5  .global s2_ptr
6  .global test_pattern
7
8  led_ptr:      .word 0x10000000
9  s1_ptr:       .word 0x10000004
10 s2_ptr:       .word 0x10000004
11 test_pattern: .word 0x000000AA    # Data mask used by boundary-write tests
12 .end
```